Event-Driven Chess Game Service: Architecture Documentation

Version 0.2 (draft). Single-owner-per-game design: Gateway, Game Service shards, Persistence Writer, event broker, Redis and PostgreSQL.

1. Overview

This system runs online chess games. Every game has one owner: a worker that keeps the game in memory. When a player makes a move, this worker checks the move, updates the clocks and sends the reply right away. Saving the game to the database happens afterwards, in the background.

The event broker (a message system between services) is not used for moves. It only sends news about finished moves to slower helpers, such as ratings, archives, notifications and engine analysis.

The goals are:

- **Fast moves.** A move is one step inside one program. It does not wait for a queue or a database.
- **No clashes.** Only one worker changes a game, so two updates can never fight over it.
- **Easy recovery.** Every move is saved in a move log. After a crash, a game can be rebuilt from the log.
- **Separate helpers.** Slow jobs listen for events and can grow on their own.

What changed from v0.1

Topic	Before (v0.1, four queues)	Now (v0.2, one owner per game)
Queue steps per move	4	0 (one message inside the program)
Redis use before the reply	Several reads and writes	None; the live game is in memory
Database write before the reply	Yes	No; it happens later, move log first
Move checking	Separate Evaluator service	Part of the game worker

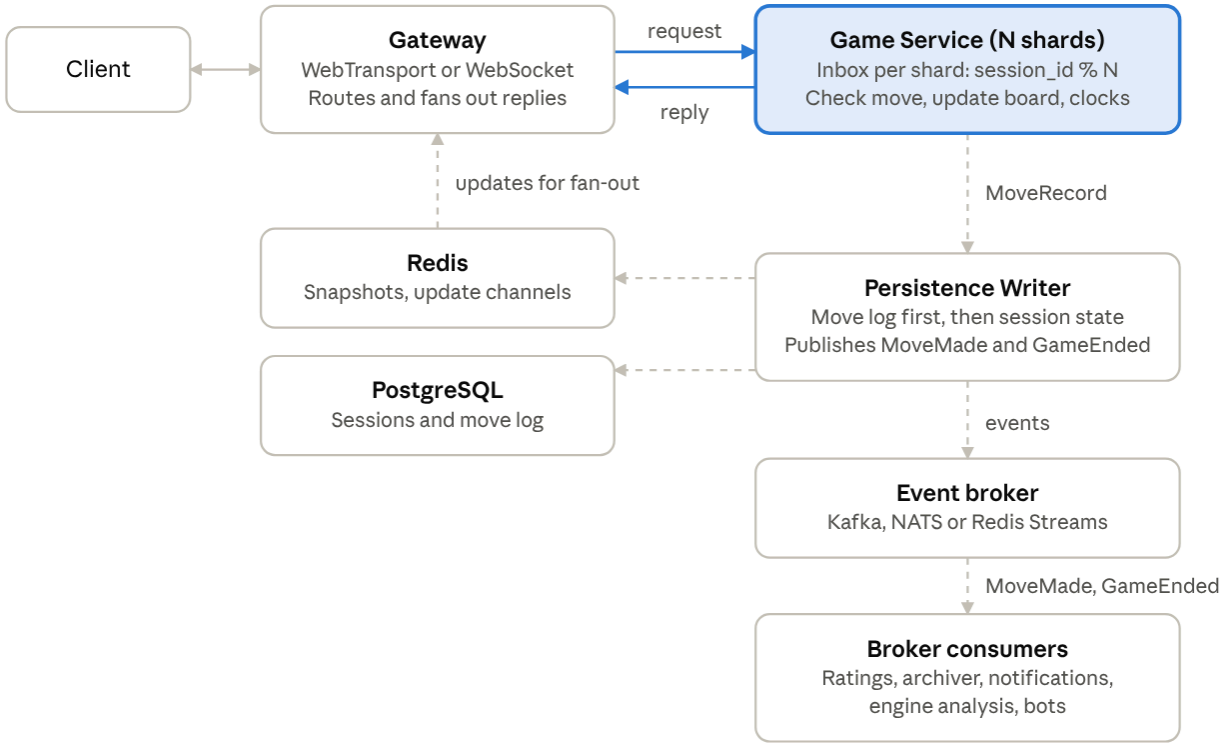
Topic	Before (v0.1, four queues)	Now (v0.2, one owner per game)
Session handling	Separate Manager service	Part of the Game Service and the Persistence Writer
Idle queue	Needed to load old games	Not needed; the worker loads an old game when it needs it

2. Architecture at a glance

The Gateway and one shard answer each move. Saving and events happen after the reply.

One worker answers each move; saving and events follow

Highlighted: the main path that answers a move. Dashed: work done after the reply.



architecture · 8 components, 1 hot path, async persistence and events

The highlighted path is all a move waits for: Gateway, shard, reply. Saving, Redis updates and broker helpers run after it.

3. Components

The system has a gateway, game workers, a persistence writer, an event broker and two data stores.

Part	What it is	What it does
Gateway	A service that holds no game data	Keeps player connections open, sends each request to the right worker, sends replies to players and spectators
Game Service	Workers that keep games in memory	Check moves, update the board and clocks, reply, create events
Persistence Writer	A background worker	Saves each move and the game state to PostgreSQL, and copies the state to Redis
Event broker	A message system	Carries news (events) to slower helpers. Options: Kafka, NATS JetStream, Redis Streams
Redis	A fast store	Keeps game copies for recovery and passes live updates to the Gateway
PostgreSQL	The main database	Stores games and the move log for good

3.1 Gateway

- Keeps each player's connection open. It uses WebTransport (over QUIC) or WebSocket (see section 9). Each connection gets a `conn_id`.
- Sends each request to the right shard. A shard is a small group of games handled by one worker. The shard number is `session_id % N`, which means the remainder when you divide by N, the number of shards.
- With many servers, it picks the owner server using consistent hashing (see section 9).
- Listens for game updates and sends each reply to both players and any spectators.

3.2 Game Service

- Runs N shards. Each shard has its own inbox, which is a queue inside the program (for example a tokio `mpsc` channel in Rust).
- Keeps its live games in memory, for example in a `HashMap<u64, GameSession>`. Only this shard may change those games.
- If a request is for a game the shard does not have, it loads the game from PostgreSQL once. If the game does not exist, it creates it. This is called a cold session.
- For every request it checks the action, applies the move, updates the clocks, replies through the Gateway and sends a `MoveRecord` to the persistence channel.

3.3 Persistence Writer

- Reads from the persistence channel and adds each move to the PostgreSQL move log first.
- Saves the main game fields to PostgreSQL in batches and puts a copy in Redis.
- After a move is saved, it publishes `MoveMade` and `GameEnded` events to the broker.

3.4 Broker helpers

Ratings, move archiving, notifications, engine analysis and bots listen for broker events. They are not part of the move path, so a slow helper never slows down a move.

4. Request flow

A move is answered from memory. Saving and events come after.

4.1 Main path (the game is already in the shard)

1. The player's client sends a request to the Gateway.
2. The Gateway puts it in the inbox of shard `session_id % N`.
3. The game worker checks the action, applies the move and updates the clocks, all in memory.
4. The worker sends the reply through the Gateway to both players and any spectators.
5. The worker sends a `MoveRecord` to the persistence channel.
6. The Persistence Writer adds the move to the log, saves the game state, copies it to Redis and publishes `MoveMade` to the broker.

The reply in step 4 does not wait for steps 5 and 6.

4.2 Cold session (the game is not in the shard)

1. The worker looks for `session_id` in its memory and does not find it.
2. It loads the game from PostgreSQL (or from Redis, if the copy there is newer), or creates a new game.
3. It keeps the game in memory and continues from step 3 of the main path.

Requests for the same game that arrive during loading wait in the inbox. This keeps them in order and stops the game from being loaded twice.

4.3 Illegal move

The worker leaves the game as it is, writes nothing to the move log and sends an error back to the player. No event is published.

5. Messages and events

Three kinds of messages move inside the program on the main path. The broker only carries events.

5.1 Request (Gateway to shard)

The old shape `{ id, move_or_select, role }` was unclear. The new shape says exactly what the player wants. It also adds fields for routing and for safe retries.

```
pub enum Action {
    Select { square: u64 },        // one square, as a bitboard with one bit
set
    Move   { from: u64, to: u64 }, // two squares, each with one bit set
}

pub struct Request {
    pub session_id: u64, // which game
    pub conn_id: u64,    // which connection to reply on
    pub seq: u32,        // request number for this game, used to ignore
repeats
    pub role: u8,        // 0 = white, 1 = black
    pub action: Action,
}
```

5.2 Reply (shard to Gateway)

The reply holds `session_id`, `conn_id`, `seq`, what the client needs to draw the game (bitboards, `selected`, `moves_available`, `status`, `side_to_move`, clocks) and an error if the action was illegal.

5.3 Save record (shard to Persistence Writer)

```
pub struct MoveRecord {
    pub session_id: u64,
    pub seq: u32,
    pub role: u8,
    pub from: u64,
    pub to: u64,
    pub new_state: GameSession, // only the fields that must be saved
}
```

5.4 Broker events

Event	Sent by	Heard by	Meaning
`MoveMade`	Persistence Writer	Spectators, archiver, analysis	A move has been saved
`GameEnded`	Persistence Writer	Ratings, notifications, archiver	The game is over
`AnalysisRequested`	Gateway or Game Service	Engine analysis workers	A player or bot asked for an engine opinion

6. Data model

The live `GameSession` is kept in the worker's memory. Redis and PostgreSQL keep copies for recovery and for permanent storage.

6.1 GameSession

Field	Type	Meaning	Saved in
`id`	`u64`	Game ID	Memory, Redis, PostgreSQL
`bitboards`	`Boards`	Where each piece is	Memory, Redis, PostgreSQL
`mode`	`mode`	Game mode	Memory, Redis, PostgreSQL
`side_to_move`	`u8`	Whose turn it is: 0 = white, 1 = black	Memory, Redis, PostgreSQL
`castling`	`String`	Who can still castle, for example `KQkq` or `-`	Memory, Redis, PostgreSQL
`ep_target`	`u64`	Square where en passant is possible	Memory, Redis, PostgreSQL
`halfmove`	`u32`	Half-moves since the last capture or pawn move	Memory, Redis, PostgreSQL
`fullmove`	`u32`	Move number	Memory, Redis, PostgreSQL

Field	Type	Meaning	Saved in
`status`	`GameStatus`	Ongoing, check, checkmate, draw and so on	Memory, Redis, PostgreSQL
`piece_selected`	`Option<Piece>`	The piece the player picked	Memory only
`selected`	`u64`	The square the player picked	Memory only
`moves_available`	`u64`	Squares that piece can legally move to	Memory only

`piece_selected`, `selected` and `moves_available` only matter while a player is choosing a move. They can be worked out again at any time, so they stay in memory and are never saved to Redis or PostgreSQL.

6.2 Stores

- **Redis.** Holds a copy called `session:{id}`, written by the Persistence Writer. It also has update channels that the Gateway uses to send updates to players. Each copy expires after a set time (TTL). Before a copy is removed, the writer saves the main fields to PostgreSQL.
- **PostgreSQL `sessions`.** The main fields of each game.
- **PostgreSQL `moves`.** A move log that only grows. Each row has `session_id`, `seq`, `role`, `from`, `to` and a time. A unique rule on `(session_id, seq)` means saving the same move twice does nothing.

7. Order, safety and clocks

Each game has one owner worker. The order of moves comes from the shard inbox, so no locks are needed.

- **One writer.** A game belongs to shard `session_id % N`. Only that shard's worker reads and changes the live game.
- **The inbox keeps order.** Requests for a game are handled in the order they arrive, even if the game is still loading.
- **Request numbers.** Each request has a `seq` number for its game. If the worker has already applied a request, it ignores it. This makes retries safe.
- **Turn check.** The worker rejects a move if `role` is not the side whose turn it is (`side_to_move`).
- **Clocks on the server.** The same worker runs the clocks. The move, the clock update and the reply happen as one step, with no other service involved.

- **Safe saving.** The unique rule on `(session_id, seq)` in the move log means a repeated save does nothing.

8. Saving and failures

The reply is sent after the update in memory, so safety depends on the move log. The risk is that a crash can lose the last few moves. The options below reduce that risk.

8.1 Saving options

Option	Reply is sent after	What a crash can lose	Cost
Save later (default)	The update in memory	The last few moves	Fastest
Log first	The move is added to the log	Only a move that was never confirmed	One log write before each reply
Log in batches	A batch of moves is written	Moves in the batch that is still open	Fewer writes, a small delay

8.2 What happens when something fails

Problem	What happens
A game worker crashes	The shard restarts. Each game is loaded again when its next request arrives, from Redis or PostgreSQL
The Persistence Writer crashes	It continues from where it stopped. The unique rule on `(session_id, seq)` ignores repeats
PostgreSQL is down	The writer retries with longer and longer waits. Games continue in memory while the backlog grows, so set an alert and a limit
Redis is down	Moves still work. The Gateway sends replies directly and loads games from PostgreSQL
The broker is down	Helpers fall behind, but moves still work. The writer sends the missed events again from the move log when the broker is back
A player's connection drops	Replies for that `conn_id` are dropped. The client asks for the game state again when it reconnects

Problem	What happens
A broken event	It goes to a dead-letter topic (a place for failed events) after N tries

9. Connections and growth

Use WebTransport over QUIC, with WebSocket as a backup. Send each game to one owner server using consistent hashing.

9.1 Connections

- **Main choice: QUIC.** Native apps can use QUIC directly. Browsers use WebTransport, which runs on HTTP/3. QUIC connects faster, does not let one lost packet block other streams, and keeps the connection when the player changes network.
- **Backup: WebSocket.** Some networks block UDP, and browser support for WebTransport is not the same everywhere. The Gateway must accept both and treat them the same way.
- **Streams.** Use separate streams for moves, clock ticks and chat so one cannot block another.
- **Reliability.** Moves must use reliable, ordered streams. Unreliable datagrams are only fine for things like clock sync.
- **What to expect.** A move is only a few bytes, so the connection type saves a few milliseconds on bad networks. The bigger gain comes from removing queues and database writes from the move path (section 4).

9.2 Growth

- **One server.** Add more shards. Each shard has its own inbox.
- **Many servers.** The Gateway (or a load balancer with sticky routing) uses consistent hashing to pick the owner server for each `session_id`. Consistent hashing means few games change owner when servers are added or removed. Each server runs its own shards.
- **Gateway.** Add more Gateway instances. Each one listens to the update channels for the games its players are in.
- **Persistence Writer.** Split the work by `session_id` so one writer handles the log of each game.
- **Broker helpers.** Use consumer groups (several copies of one helper that share the work). They grow separately from the move path.

- **Moving a game.** When servers are added or removed, a game moves only after its old owner saves its log and the new owner loads the game.

10. Monitoring

- Add a trace ID to every request and pass it through the Gateway, the worker, the Persistence Writer and every event.
- Measure move time (request received to reply sent), how full each shard inbox is, and how far behind saving is.
- Measure how far behind each broker helper is.
- Count illegal moves, ignored repeats, cold loads and reconnects.

11. Open questions

1. **How much can we afford to lose?** Pick "save later", "log first" or "log in batches" (section 8.1).
2. **Which broker?** Pick Kafka, NATS JetStream or Redis Streams for the slower helpers.
3. **One server or many?** Decide whether version 1 needs consistent hashing from the start.
4. **Is WebTransport ready?** Check browser support before using it as the main browser connection.
5. **When do games leave memory?** Decide how long a Redis copy lives (TTL) and when idle games leave worker memory.
6. **The fifth queue.** The first description said five queues but listed four. If a dead-letter or retry queue was meant, decide where it fits in the new design.

12. Glossary

- **Shard.** A small group of games, handled by one worker with its own inbox. A game's shard is `session_id % N`.
- **Game worker.** The code in a shard that checks and applies moves.
- **Cold session.** A game that is not in memory and must be loaded from storage first.
- **Save later (write-behind).** Reply first, save afterwards.
- **Move log.** A list of all moves that only grows. It is used to rebuild a game.
- **Consistent hashing.** A way to pick a server for each game so that few games move when servers change.
- **Event broker.** A message system that passes events from one service to others.

13. Tools and technology

The tools below are the ones named in this project. "Chosen" means you picked it. "Suggested" means it was proposed in this document and still needs your approval. "Open" means no decision has been made.

Tool	What it is used for	Status
Rust	The language for the services. The `GameSession` code is already written in Rust	Chosen
tokio	Runs the shards and their inboxes (`mpsc` channels)	Suggested
QUIC	The main connection between players and the Gateway	Chosen
WebTransport	How browsers use QUIC (it runs on HTTP/3)	Suggested
WebSocket	The backup connection when UDP or WebTransport is not available	Suggested
Redis	Game copies and live update channels	Chosen
PostgreSQL	Games and the move log	Chosen
Event broker (Kafka, NATS JetStream or Redis Streams)	Carries events to the slower helpers	Open (see section 11)
Chess engine such as Stockfish	Engine analysis workers, kept outside the shards	Suggested
Monitoring tools	Trace IDs, move time and queue sizes (section 10)	Open